SkillSync: Explainable, Efficient, and Transparent Resume Evaluation

Maximilian Schulten [1] Jack Alberto [1] Tommaso Beretta [1] William Hugus [1] Nathaniel Hahn [1] Dr. Shaui Xu[1]

[1]Case Western Reserve University School of Engineering, Department of Computer and Data Sciences

CASE WESTERN RESERVE UNIVERSITY
Case School of Engineering

Abstract

Job searching is often a time consuming and inefficient process that relies heavily on user judgement to assess their own fit for each job opening. With the rise of online job boards that everyone can access, the online job market has become overly competitive in recent years. This project, SkillSync, develops a lightweight, privacy focused, and data driven system for matching resumes to job descriptions.

The goal of this project is to automate and improve the user experience of job searching by generating a list of job openings for which the user is most qualified for. The system operates through a pipeline that includes skill extraction from the resume, job data aggregation, matching process, and generating the list of top matches.

System Outline

We divide **SkillSync** into four main steps:

1. **Skill Extraction:** The user uploads a resume and the skills are extracted while personal identifiable information is removed.
2. **Job Aggregation:** We query multiple APIs to maximize job opportunities; each job has key features extracted.
3. **Skill Matching:** The skills extracted during step on are matched with the job features extracted in step two. Keyword search and a matching model are applied.
4. **Match Presentation:** We present the ordered top matches to the user in a clean and functional UI.

Skill Extraction and Preprocesssing

The skill extraction modules contain the data cleaning, resume classifier and skill extraction, the result of this module is a full understanding of the users resume without and personally identifying information (PII).

- **Data Cleaning:**
  - strips non-alphanumeric characters and white space
  - uses regex and named entity recognition to obsfucate PII
  - Removing PII eliminates implicit bias in later trained networks
- **Resume Classification:**
  - Linear support vector matchine classifies resumes into 12 industry buckets
  - Embedding transformer to vectorize the resume
  - Achieves >90% top-2 accuracy
- **Skill Extraction:**
  - spaCy PhraseMatcher used for rapid dictionary matching agaisnt 99k relevant terms
  - GLiNER (bidirectional transformer) used to extract semantic information like education and experience

References and Relevant Classes

CSDS 132, CSDS 133, CSDS 221, CSDS 233, CSDS 293, CSDS 310, CSDS 338, CSDS 240, CSDS 356, CSDS 391

[1] Adzuna Ltd. Adzuna Job Search API v1, 2024.

[2] Daily Muse Inc. The Muse API, 2024.

[3] European Commission. European Skills, Competences, Qualifications and Occupations (ESCO), 2023. European Commission DG Employment, Social Affairs and Inclusion.

[4] Matthew Honnibal, Ines Montani, Sofie Van Landeghem, and Adriane Boyd. spaCy: Industrial-strength Natural Language Processing in Python, 2020.

[5] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[6] U.S. Department of Labor, Employment and Training Administration. O*NET 28.0 Database, 2023. National Center for O*NET Development.

[7] U.S. Office of Personnel Management. USAJobs Search API, 2024.

System Architecture

SkillSync is built as a three-tier web application. The front end is a single-page interface where users paste their resume and receive a ranked list of job matches. It communicates with a Flask back end that exposes two REST endpoints, /find-jobs for full pipeline execution and /score-job for scoring a single listing. The back end orchestrates a multi-stage NLP pipeline: resume parsing and classification, job aggregation, skill extraction, and composite scoring. All components are containerized via Docker with memory limits set to the resource demands of the embedded transformer models.

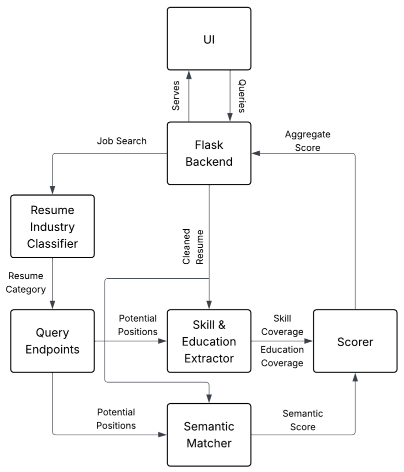

Figure 1. System Architecture

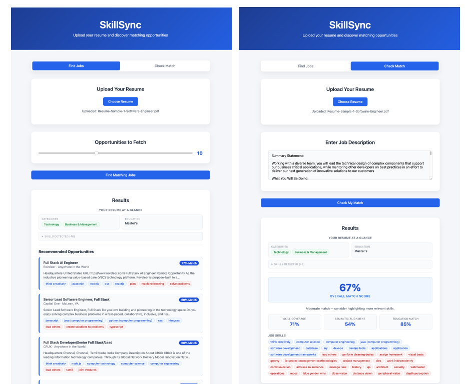

Figure 2. Front End Examples

Data Aggregation

- Asynchronous fetching from 8 seperate job boards, covering multiple sectors and industries
- Data is normalized into a Job Object from its original schema
- Jobs are filtered and duplicates removed to ensure unique listings
- Skill extraction applied to jobs

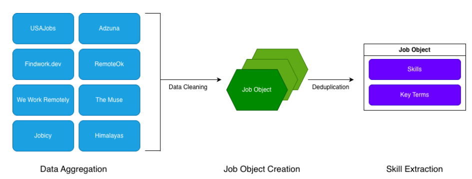

Figure 3. Data Aggregation Pipeline

Front End

The interface is minimal and clean, featuring a resume text input and a results panel. After submission, users see each job ranked by composite match score, with per-job breakdowns of matched skills, unmatched skill gaps, and sub-scores for skill coverage, semantic similarity, and education alignment. This transparency lets users understand why a job ranked highly rather than treating the score as a black box.

Matching Model

SkillSync scores each job listing against a parsed resume using a convex combination of three complementary signals:

$$S_{\text{total}} = \alpha \cdot S_{\text{skill}} + \beta \cdot S_{\text{sem}} + \gamma \cdot S_{\text{edu}}, \quad \alpha + \beta + \gamma = 1 \tag{1}$$

Default weights: $\alpha = \beta = 0.45, \gamma = 0.1$.

- **Skill Coverage (S_{skill})**
  Matches deterministically from databases aggregated from O*NET and ESCOU alongside a fuzzy matching system leveraging `all-MiniLM-L6-v2` embeddings and cosine similarity:

$$S_{\text{skill}} = \tilde{\sigma}\left(\frac{1}{|J|}\sum_{j \in J} w\left(\max_{r \in R} \cos(e_j, e_r)\right)\right) \tag{2}$$

  where $w(x)$ rewards matches above threshold $\tau = 0.6$, and $\tilde{\sigma}$ applies a shifted sigmoid function to ensure saturation at acceptable raw coverage values.
- **Semantic Similarity (S_{sem})**
  To capture contextual alignment beyond discrete skills, the resume and job description are each segmented into overlapping windows of 30 tokens with stride 10. The mean of per-window maximum cosine similarities $\bar{s}$ is mapped to a calibrated score via a normal CDF fit empirically to 25k+ job–resume pairs ($\mu = 0.345,\ \sigma = 0.067$):

$$S_{\text{sem}} = \Phi_{\mu,\sigma}(\bar{s}), \quad \bar{s} = \frac{1}{|W_J|}\sum_{w \in W_J} \max_{v \in W_R} \cos(e_w, e_v) \tag{3}$$

- **Education Coverage (S_{edu})**
  Combines degree-level alignment and major similarity:

$$S_{\text{edu}} = 0.7 \cdot S_{\text{deg}} + 0.3 \cdot S_{\text{maj}} \tag{4}$$

S_{deg} is a hierarchical score (1.0 if the candidate meets or exceeds the required degree level, 0.5 if one level below, 0.0 otherwise). S_{maj} is the cosine similarity between embedded major strings, thresholded at $\tau_{\text{edu}} = 0.6$

Testing and Results

Our project features a full pytest suite meant to ensure consistent development among a multi-person team and deployment verification. These tests provide coverage across our entire build, preprocess, extract and inference modules allowing us to verify changes and catch bugs.

An example of our results are seen in the front end demo, along with the figure below, which describes the approximate distribution of our pipeline on a random set of 25,800 resume-job combinations.

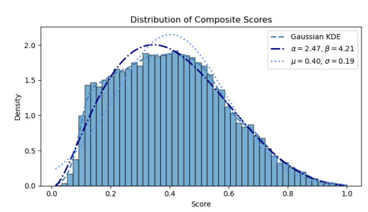

Figure 4. Score Distribution

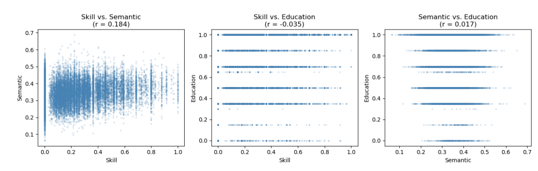

Figure 5. Scoring Correlates

Project Challenges

[jia15, txb341, wmh42, mls384, nrh51, sxx214]@case.edu